1. #include <stdio.h>

#include <signal.h>

#include <unistd.h>

void handler(int sig)

{

printf("\nSIGINT received!\n");

printf("Signal number = %d\n", sig);

}

int main()

{

signal(SIGINT, handler);

while (1)

{

printf("Program is running...\n");

sleep(2);

}

return 0;

}

```
hamsini@HamsiniVanam:~$ gcc signal.c -o signal
hamsini@HamsiniVanam:~$ ./signal
Program is running...
Program is running...
Program is running...
^C
SIGINT received!
Signal number = 2
Program is running...
^C
SIGINT received!
Signal number = 2
Program is running...
Program is running...
^\Quit
hamsini@HamsiniVanam:~$
```

2. #include <stdio.h>

#include <signal.h>

#include <unistd.h>

void handler(int sig)

{

if (sig == SIGTERM)

printf("\nSIGTERM received!\n");

}

int main()

{

signal(SIGTERM, handler);

printf("Process ID: %d\n", getpid());

printf("Waiting for SIGTERM...\n");

while (1)

{

sleep(2);

```
}

return 0;

}
```

```
hamsini@HamsiniVanam:~$ gcc sigterm.c -o sigterm
hamsini@HamsiniVanam:~$ ./sigterm &
[1] 122
hamsini@HamsiniVanam:~$ Process ID: 122
Waiting for SIGTERM...
kill -SIGTERM 122
hamsini@HamsiniVanam:~$
SIGTERM received!
kill -9 122
hamsini@HamsiniVanam:~$
[1]+  Killed                  ./sigterm
hamsini@HamsiniVanam:~$
```

```
3. #include <stdio.h>

#include <signal.h>

#include <unistd.h>

#include <sys/wait.h>


void child_handler(int sig)

{

printf("SIGCHLD received: Child has terminated\n");

wait(NULL);

}


int main()

{

signal(SIGCHLD, child_handler);


pid_t pid = fork();


if (pid == 0)

{
```

```
printf("Child is running...\n");
sleep(2);
printf("Child is exiting...\n");
}
else
{
printf("Parent is waiting...\n");
sleep(5);
printf("Parent finished.\n");
}

return 0;
}
```

```
hamsini@HamsiniVanam:~$ gcc sigchild.c -o sigchild
hamsini@HamsiniVanam:~$ ./sigchild
Child is running...
Parent is waiting...
Child is exiting...
SIGCHLD received: Child has terminated
Parent finished.
hamsini@HamsiniVanam:~$
```

4.
```
#include <stdio.h>
#include <unistd.h>
#include <signal.h>
#include <stdlib.h>

void handle_sigpipe(int sig)
{
printf("\nSIGPIPE received!\n");
printf("The reading end of the pipe is closed.\n");
exit(1);
}
```

```
int main()
{
int fd[2];

signal(SIGPIPE, handle_sigpipe);

pipe(fd);

printf("Pipe created.\n");

// Close the reading end
close(fd[0]);

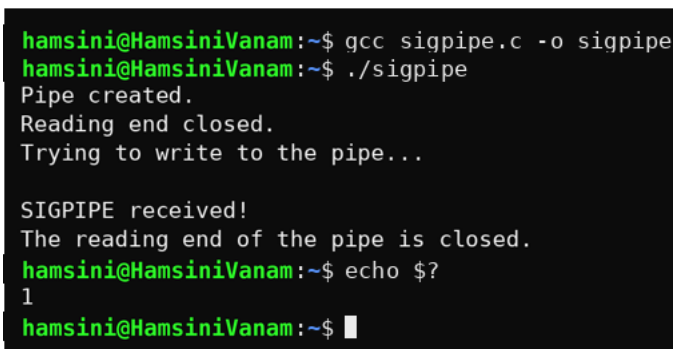
printf("Reading end closed.\n");

printf("Trying to write to the pipe...\n");

write(fd[1], "Hello", 5);

printf("Write completed.\n");

return 0;
}
```




5. **Program**

```
#include <stdio.h>
```

```
#include <signal.h>
#include <stdlib.h>

void handle_sigsegv(int sig)
{
printf("\nSIGSEGV received!\n");
printf("Invalid memory access detected.\n");
exit(1);
}

int main()
{
signal(SIGSEGV, handle_sigsegv);

printf("Program started.\n");

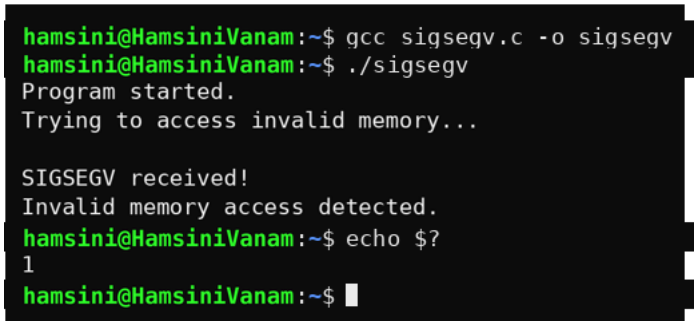
int *p = NULL;

printf("Trying to access invalid memory...\n");

*p = 10;

printf("Program completed.\n");

return 0;
}
```


hamsini@HamsiniVanam:~$ gcc sigsegv.c -o sigsegv
hamsini@HamsiniVanam:~$ ./sigsegv
Program started.
Trying to access invalid memory...
SIGSEGV received!
Invalid memory access detected.
hamsini@HamsiniVanam:~$ echo $?
1
hamsini@HamsiniVanam:~$